# 1. What is CSS

## 1.1 The role of CSS in the web

The web is built from three main languages:

- **HTML** – what is on the page (structure & content)
- **CSS** – how it looks (design & layout)
- **JavaScript** – how it behaves (logic & interactions)”

HTML by itself is like a Microsoft Word document with no formatting: just black text on a white background, stacked one after another.

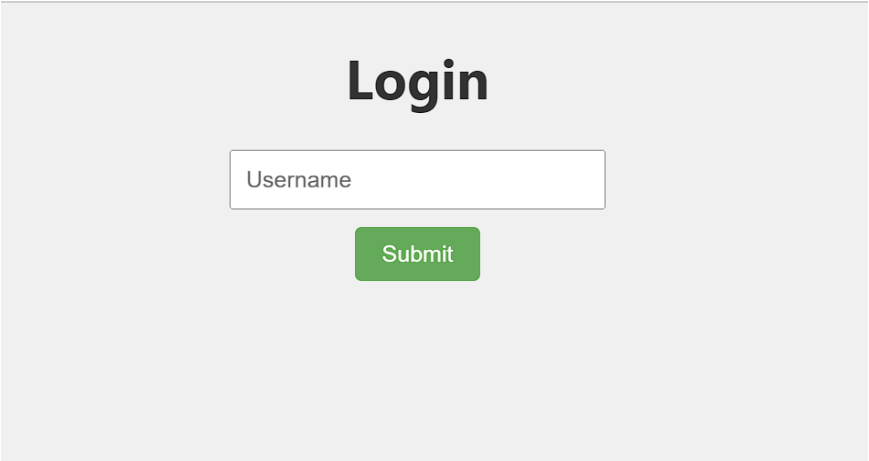
CSS changes that by controlling:

- **Colors** – of text, backgrounds, borders
- **Typography** – fonts, sizes, bold, italics, letter spacing, line height
- **Layout** – where elements sit: side-by-side, centered, grid, columns
- **Spacing** – margins, paddings, gaps
- **Shapes** – rounded corners, circles, card designs
- **Visual feedback** – hover states, focus styles
- **Animations & transitions** – smooth visual changes

CSS is what turns this:

# Login

Username Submit




Now the experience is:

- Easier to read
- More trustworthy
- More usable

This is **why CSS is helpful**: it makes the web usable and pleasant.

# 2. CSS Syntax

## 2.1 CSS Rule Structure

A CSS rule always follows this shape:

```
None
selector {
  property: value;
  property: value;
}
```

- **Selector** → Which element(s) in the HTML we want to style
- **Property** → What aspect we want to change (color, font-size, margin…)
- **Value** → How we want it to look (red, 16px, center…)

Example:

```
None
p {
  color: red;
  font-size: 18px;
}
```

Meaning:

- “Select all `<p>` elements”, then
- “Make their text color red”, and
- “Make their font size 18 pixels”.

## 2.2 Different types of selectors

**Element selector**

Targets all HTML tags of that type:

```
None
h1 {
  color: blue;
}
```

“All `<h1>` elements are blue.”

**Class selector**

Use class for reusable styles:

```
None
.highlight {
  background-color: yellow;
}
```

HTML:

```
None
<p class="highlight">Important text</p>
```

“All elements with `class="highlight"` will have yellow background.”

**ID selector**

IDs are meant to be **unique** per page:

```
None
#main-title {
  font-size: 32px;
}
```

HTML:

```
None
<h1 id="main-title">Home</h1>
```

Use for unique things (like one main section). Don't overuse IDs for styling, they have very high specificity (we'll get to that).

## 2.3 Declarations and units

Inside a rule we have declarations:

```
None
p {
  font-size: 16px; /* px = pixel */
  margin-top: 10px;
  line-height: 1.5; /* unitless multiplier of font-size */
}
```

Common units:

- `px` – fixed pixels
- `%` – relative to parent
- `rem` – relative to root font size (usually better for accessibility)
- `em` – relative to parent font size

## 2.4 Comments in CSS

```
None
/* This is a CSS comment */
p {
  color: blue; /* this affects paragraph text color */
}
```

Comments help them explain their own code.

# 3. How We Connect CSS to HTML

Once they know the syntax, now: **Where do we put this CSS? How does the browser see it?**

There are 3 integration methods:

## 3.1 Inline CSS

CSS is written inside the HTML element using the `style` attribute.

```
None
<h1 style="color: red; font-size: 32px;">Hello</h1>
```

Here:

- The browser reads the HTML node `<h1 ...>`
- Sees `style="..."`
- Applies these styles **only** to this exact element

**Pros:**

- Very simple to understand
- Good for quick tests

**Cons (important for them to know):**

- Not reusable
- Hard to maintain: if you want to change all headings, you must change many tags
- Mixes structure with design → messy code
- Very high specificity (hard to override later)

“Inline CSS is like writing notes directly on the furniture with a marker – it works, but it’s not how you design a house.”

## 3.2 Internal CSS (`<style>` tag in `<head>`)

We keep CSS inside the HTML file, but in a separate `<style>` block.

```
None
<!DOCTYPE html>
<html>
  <head>
    <title>Internal CSS Example</title>
    <style>
      h1 {
        color: green;
      }

      p {
        color: #555;
      }
    </style>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>This page uses internal CSS.</p>
  </body>
</html>
```

The browser:

1. Loads HTML
2. Reads `<style>` inside `<head>`
3. Applies these rules to elements

**Pros:**

- CSS is in one place (not scattered as inline)
- Good for small single-page demos
- Easy for teaching

**Cons:**

- If you have 10 pages, you must copy the same `<style>` block into each page
- Harder to share styles across many pages
- Less scalable for real apps

## 3.3 External CSS (professional way)

We put all CSS in a separate file, e.g., `styles.css` Then we **link** it from HTML.

**HTML:**

```
None
<!DOCTYPE html>
<html>
  <head>
    <title>External CSS Example</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <h1>Welcome</h1>
    <p>This page uses external CSS.</p>
  </body>
</html>
```

**styles.css:**

```
None
h1 {
  color: navy;
}

p {
  color: #444;
}
```

The browser:

1. Loads HTML
2. Finds `<link rel="stylesheet" href="styles.css">`
3. Fetches `styles.css` as a separate file
4. Applies its rules to the HTML

**Why this is the preferred, professional method:**

- Separation of concerns: HTML (content) vs CSS (design)
- One CSS file can style **many pages**
- Changing one file updates the entire site
- Easier for teams (one person can work on CSS, others on HTML/JS)
- Better caching & performance (CSS file is downloaded once and reused)

You can emphasize:

“In real projects, almost all CSS is external. Inline is for quick tests, internal is for small examples and learning.”

# 4. Specificity & Cascading

What happens when **two rules target the same element**?

## 4.1 What is specificity?

Specificity is a scoring system. The more “specific” the selector, the higher its priority when rules conflict.

Rough idea:

- Inline style – highest
- ID selector – very strong
- Class / attribute / pseudo-class – medium
- Element selector – weak

Example:

```
None
<p id="info" class="highlight">Hello</p>
```

```
None
p {
  color: blue;
}

.highlight {
  color: green;
}

#info {
  color: red;
}
```

Which color wins?

- `p` says blue
- `.highlight` says green
- `#info` says red

Result = **red**, because ID > class > element.

“CSS is like a manager listening to three people:

- A general rule (element)
- A more specific group (class)
- A direct instruction (ID) The direct instruction (ID) wins.”

## 4.2 Order in the file (last one wins)

If specificity is equal, the **last rule in the CSS file** wins.

```
None
p {
  color: blue;
}

p {
  color: orange;
}
```

Same selector → same specificity → second rule (orange) wins.

## 4.3 Inline vs internal vs external

If all else is equal:

- Inline styles (inside `style=""`) override internal and external
- Then internal `<style>`
- Then external CSS

That’s why we avoid inline: it’s too strong.

## 4.4 Inheritance (part of the cascade)

Some CSS properties are **inherited automatically** from parent to children.

Examples (inherited):

- `color`
- `font-family`
- `line-height`

Non-inherited examples:

- `margin`
- `padding`
- `border`

- `width`, `height`

Example:

```
None
body {
  color: #333;
  font-family: Arial, sans-serif;
}
```

All text inside `<body>` inherits this color and font unless overridden.

This explains why sometimes changing `body` is easier than styling each element individually.

# 5. Basic CSS Properties (Color, Background, Typography, Spacing)

## 5.1 Color

Text color:

```
None
p {
  color: red;
}

h1 {
  color: #3498db; /* hex */
}

a {
  color: rgb(0, 128, 0); /* rgb */
}
```

Background color:

```
None
div {
```

```
  background-color: #f0f0f0;
}
```

## 5.2 Background images

```
None
.hero {
  background-image: url("hero.jpg");
  background-size: cover; /* fill and crop */
  background-position: center; /* center cropping */
}
```

## 5.3 Typography

```
None
body {
  font-family: Arial, Helvetica, sans-serif;
  font-size: 16px; /* base size */
  line-height: 1.5; /* space between lines */
}

h1 {
  font-size: 2.5rem; /* 2.5 * root font-size */
  font-weight: bold;
  text-align: center;
}

p {
  text-align: justify;
}
```

## 5.4 Spacing with margin & padding

```
None
.box {
  margin: 20px; /* outside space */
  padding: 15px; /* inside space */
}
```

# 6. Box Model – Deep Explanation

Now we go deeper: every element is a box and spacing has layers.

## 6.1 What is the box model?

Every element is a rectangle composed of:

1. **Content** → text, image, etc.
2. **Padding** → transparent area around content, inside the box
3. **Border** → line around padding + content
4. **Margin** → transparent area outside border

Code example:

```
None
.card {
  width: 300px; /* content width (by default) */
  padding: 20px; /* inner spacing */
  border: 2px solid #333; /* outer line */
  margin: 30px; /* distance from other elements */
  background-color: #fafafa;
}
```

“Is the total width really 300px?”

No. Total width by default:

total width = content width + padding-left + padding-right + border-left + border-right

Here:

- content width = 300
- padding = 20 + 20 = 40
- border = 2 + 2 = 4

Total width = **344px**

This explains many “why is my layout overflowing?” bugs.

## 6.2 `box-sizing: border-box`

To make life easier, professionals often use:

```
None
* {
  box-sizing: border-box;
}
```

Now:

- `width: 300px` means **content + padding + border together = 300px**

This is much easier when building grids and responsive layouts.

## 6.3 Why box model matters

- Determines real element size
- Affects how layouts fit inside containers
- Helps debug UI issues (overflow, misalignment)
- Essential for Flexbox, Grid, and serious layout systems

.

# 7. Positioning

Now that they know how big each box is, we talk about **where those boxes live**.

## 7.1 Normal flow (position: static)

By default, elements:

- Stack top to bottom
- Inline content flows left to right
- Respect margin & padding

```
None
div {
  position: static; /* default, even if you don't write it */
```

```
}
```

You **cannot** move static elements using `top`, `left`, `right`, `bottom`.

## 7.2 `position: relative`

The element stays in the **normal flow**, but you can offset it visually.

```None
.box {
  position: relative;
  top: 10px; /* move down 10px */
  left: 20px; /* move right 20px */
}
```

The important idea:

- The original space the element occupies is still reserved in the layout.
- Only the visual box is shifted.

**Main real-world use**: to act as a **positioning context** for absolutely positioned children.

## 7.3 `position: absolute`

The element is:

- Removed from normal flow (does not push siblings)
- Positioned relative to the **closest ancestor that is not static**

If no ancestor has `position: relative/absolute/fixed/sticky`, it will be positioned relative to the **page (html / body)**.

Example: sale badge on a card.

```None
<div class="product">
  <span class="badge">Sale</span>
  <h3>Product Name</h3>
```

```
</div>
```

```
None
.product {
  position: relative; /* makes this the reference for the child
*/
  width: 250px;
  padding: 20px;
  border: 1px solid #ccc;
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  background: red;
  color: white;
  padding: 5px 10px;
  border-radius: 999px;
}
```

Badge stays in the top-right corner of `.product`, not the whole page.

## 7.4 `position: fixed`

Attached to the **viewport** (screen), not to the page flow.

- Does not move when the user scrolls
- Great for sticky headers, chat buttons, “scroll to top” buttons

Example:

```
None
.chat-btn {
  position: fixed;
  bottom: 20px;
  right: 20px;
```

```
  width: 50px;
  height: 50px;
  border-radius: 50%;
  background: #2ecc71;
  color: white;
}
```

## 7.5 `position: sticky`

Hybrid between `relative` and `fixed`.

- Acts like `relative` until the element's scroll position reaches a certain value
- Then becomes `fixed` (sticks in place)

Example: sticky navigation bar:

```
None
nav {
  position: sticky;
  top: 0;
  background: white;
  z-index: 1000;
}
```

It scrolls normally until its top reaches the top of the viewport, then stays there.

## 7.6 `z-index` (stacking order)

When elements overlap, `z-index` decides which is "on top".

```
None
.box1 {
  position: absolute;
  z-index: 1;
}
```

```
.box2 {
  position: absolute;
  z-index: 2; /* this appears above box1 */
}
```

# 8. Pseudo Selectors (Pseudo-Classes & Pseudo-Elements)

Now you bring interactivity and decorations.

## 8.1 Pseudo-classes – element states

They describe **a state** of an element, not a new element.

Syntax: `selector:pseudo-class`

### `:hover`

When the user's mouse is over an element:

```
None
button:hover {
  background: #333;
  color: white;
}
```

### `:active`

When the element is being pressed:

```
None
button:active {
  transform: scale(0.97); /* small press effect */
}
```

## :focus

When an input is focused (user clicked or tabbed into it):

```
None
input:focus {
  border-color: #3498db;
  outline: none;
  box-shadow: 0 0 3px #3498db;
}
```

This is crucial for accessibility and forms.

## :nth-child()

Select elements based on their index in a parent:

```
None
li:nth-child(2) {
  color: red; /* 2nd item */
}

li:nth-child(odd) {
  background: #f9f9f9; /* alternate row shading */
}
```

## 8.2 Pseudo-elements – virtual elements

They allow you to style **parts of an element** or inject **decorative content** without changing the HTML.

Syntax: `selector::pseudo-element`

Common ones:

- `::before` – inserts content before the element's content
- `::after` – inserts content after the element's content

Both require the `content` property.

## Example: add icon before title

```
None
<h2 class="title">Features</h2>
```

```
None
.title::before {
  content: "★ ";
  color: gold;
}
```

No HTML change, but visually an icon appears.

## Example: decorative underline with `::after`

```
None
<h2 class="underline">Hello</h2>
```

```
None
.underline {
  position: relative;
```

```
  display: inline-block;
}

.underline::after {
  content: "";
  position: absolute;
  left: 0;
  bottom: -4px;
  width: 100%;
  height: 3px;
  background: linear-gradient(to right, #4facfe, #00f2fe);
}
```

You create a nice line effect under the text without extra `<div>` in HTML.